TIA cheat-sheet — LLM threat intelligence (SIA)

30-sec pitch: "I built **openai-tia-prep** — a Python + SQL pipeline that ingests messy LLM logs and flags prompt injection, jailbreaks & data exfiltration, every finding mapped to OWASP LLM Top 10 + MITRE ATLAS. 800 events → 150 flagged / 217 findings, 9 detectors, 7 SQL queries incl. anomaly detection, 83 tests + CI. I red-teamed my own detectors and fixed the bugs."

SIA mission: monitor, analyze & **forecast real-world abuse**, geopolitical/strategic threats → inform safety mitigations, product decisions, partnerships. Day-to-day: **surface novel harms → structured, decision-ready intelligence** via SQL + Python + AI tools.

Numbers (seed 7): 800 events · 156 bad-ts recovered · 63 IDs synthesized · **150 flagged** · **217 findings** (61C/126H/25M/5I) · 9 detectors · 7 queries · 83 tests · OWASP: LLM01 **132**, LLM05 29, LLM07 25, LLM02 22, LLM06 9.

OWASP LLM TOP 10 — 2025

01 Prompt Injection	**02** Sensitive Info Disclosure
03 Supply Chain	**04** Data & Model Poisoning
05 Improper Output Handling	**06** Excessive Agency
07 System Prompt Leakage	**08** Vector/Embedding Weaknesses
09 Misinformation	**10** Unbounded Consumption

Repo covers (detectors) 01/02/05/06/07; (SQL) 10 + multi-turn. Roadmap: 03/04/08/09.

MITRE ATLAS — KEY TECHNIQUES

- `AML.T0051` LLM Prompt Injection (.000 Direct / .001 Indirect)
- `AML.T0054` LLM Jailbreak · `AML.T0056` LLM Meta Prompt Extraction
- `AML.T0053` AI Agent Tool Invocation · `AML.T0057` LLM Data Leakage
- `AML.T0024` Exfiltration via AI Inference API

ATLAS = ATT&CK for ML: Recon → Model Access → Poisoning → ML Attack Staging → Exfiltration → Impact.

RAPID-FIRE DEFINITIONS

- **Direct injection**: the user types the override.
- **Indirect injection**: override rides in on ingested content (RAG/web/email/tool) — user may be innocent; worse (blurred trust boundary).
- **Jailbreak**: defeats safety/policy (persona: DAN, "developer mode").
- **Prompt leakage/extraction**: probe to reveal hidden prompt; best signal = canary token.
- **Improper output handling**: downstream trusts output → XSS/SSRF/exfil/code-exec.
- **Excessive agency**: too much tool/permission → unintended actions.
- **Model extraction**: systematic queries to steal model/boundary (LLM10/T0024).
- **Membership inference / inversion**: infer training membership / reconstruct data.
- **Denial-of-wallet**: cost-exhaustion via expensive inference (LLM10).
- **Data/model poisoning**: train-time corruption → backdoors (LLM04).

DETECTION PRINCIPLES (= YOUR TALKING POINTS)

- **Channel-awareness** — same string benign from a user, critical from a RAG doc. Provenance is part of the verdict.
- **Evasion-resistance** — normalize (NFKC, zero-width, spaced-letter, leetspeak) + decode base64/hex before matching.
- **Output-side detection** — scan the response (secret leak, exfil link), not just input.
- **Correlation > single signals** — chain injection → tool call → egress; repeat offenders; multi-turn escalation.
- **Signatures for known, anomaly (PyOD) for novel** — triage where they agree first.
- **Rank + communicate** — Finding → Mechanism → Impact → Likelihood → Recommendation → Cost of inaction.

MITIGATIONS (ONE-LINERS)

- **Injection**: trust-tag/isolate untrusted content; treat all input as hostile; instruction hierarchy.
- **Output handling**: contextual encoding; strip/allowlist markdown (kill image auto-fetch); no eval/shell/SQL on output.
- **Leakage**: canary tokens; keep secrets out of prompts; output secret-scan.
- **Excessive agency**: least-privilege tools; human-in-loop for high-impact actions.
- **Unbounded consumption**: per-key rate/token/cost budgets; max-output caps; loop guards.

THE 9 DETECTORS (REPO)

- direct / indirect / encoded **prompt injection** (LLM01)
- **jailbreak** persona override (LLM01) · **system-prompt extraction** (LLM07)
- **excessive agency** probe (LLM06)
- **sensitive disclosure** — output + inbound (LLM02)
- **improper output handling** / exfil (LLM05)

IF ASKED "WHERE DO REGEXES FAIL?"

Obfuscation (base64/unicode/zero-width/multilingual) & novel phrasings. Repo already folds unicode/spacing/leetspeak + decodes base64/hex; next layers = embedding-similarity to attack clusters, an LLM-judge classifier, canary tokens — each drops in as just another detector.